

CIC0198 - TÉCNICAS DE PROGRAMAÇÃO 2

Padrões de Projeto

Daniel Porto – daniel.porto@unb.br
Departamento de Ciência da Computação – CIC
Universidade de Brasília – UnB

Material adaptado do Prof. Marco Tulio Valente



UnB | IE | CIC

Padrões de Projeto

Introdução

Padrões

Padrões Criacionais

Padrões Estruturais

Padrões Comportamentais

INTRODUÇÃO

Design Patterns

Elements of Reusable
Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides



Cover art © 1994 M.C. Escher / Gordon Art - Baars - Holland. All rights reserved.

Foreword by Grady Booch

ADDISON-WESLEY PROFESSIONAL COMPUTING SERIES



1994, conhecido como livro da “Gangue dos Quatro” ou GoF

DESIGN PATTERNS

goodreads Home My Books Browse Community Search books

Shelves > Computer Science >

Computer Science Books

Showing 1-50 of 15,715

	Introduction to Algorithms (Hardcover) by Thomas H. Cormen (shelved 826 times as computer-science) avg rating 4.35 — 9,060 ratings — published 1989	Want to Read Rate this book ★★★★★
	The Pragmatic Programmer: From Journeyman to Master (Paperback) by Dave Thomas (Goodreads Author) (shelved 761 times as computer-science) avg rating 4.33 — 22,510 ratings — published 1999	Want to Read Rate this book ★★★★★
	Clean Code: A Handbook of Agile Software Craftsmanship (Paperback) by Robert C. Martin (shelved 752 times as computer-science) avg rating 4.37 — 22,159 ratings — published 2007	Want to Read Rate this book ★★★★★
	Structure and Interpretation of Computer Programs (Paperback) by Harold Abelson (shelved 641 times as computer-science) avg rating 4.47 — 4,739 ratings — published 1984	Want to Read Rate this book ★★★★★
	Code: The Hidden Language of Computer Hardware and Software (Paperback) by Charles Petzold (shelved 629 times as computer-science) avg rating 4.25 — 11,612 ratings — published 2000	Want to Read Rate this book ★★★★★
	Design Patterns: Elements of Reusable Object-Oriented Software (Hardcover) by Erich Gamma (shelved 532 times as computer-science) avg rating 4.20 — 11,612 ratings — published 1994	Want to Read Rate this book ★★★★★
	Algorithms to Live By: The Computer Science of Human Decisions (Hardcover) by Brian Christian (Goodreads Author) (shelved 491 times as computer-science) avg rating 4.13 — 32,445 ratings — published 2016	Want to Read Rate this book ★★★★★
	The Mythical Man-Month: Essays on Software Engineering (Paperback) by Frederick P. Brooks Jr. (shelved 454 times as computer-science) avg rating 4.01 — 14,506 ratings — published 1975	Want to Read Rate this book ★★★★★

UTILIDADE: REÚSO DE PROJETO

Suponha que eu tenho um problema de projeto

Pode existir um padrão que resolve esse problema

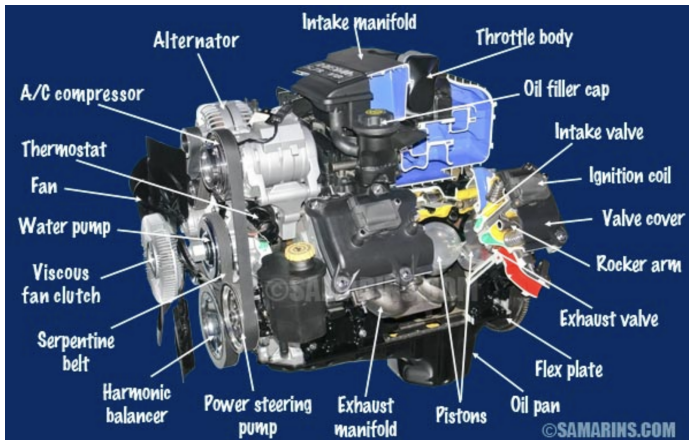
Vantagem: ganho tempo e não preciso reinventar a roda

UTILIDADE: VOCABULÁRIO PARA COMUNICAÇÃO

Vocabulário para discussões, documentação, etc

UTILIDADE: VOCABULÁRIO PARA COMUNICAÇÃO

Vocabulário para discussões, documentação, etc



Padrões de Projeto

Introdução

Padrões

Padrões Criacionais

Padrões Estruturais

Padrões Comportamentais

PADRÕES

Quais são as “peças” que posso usar no design do meu software?

Quais são os tipos de “peças” que eu posso usar:

PADRÕES

Quais são as “peças” que posso usar no design do meu software?

Quais são os tipos de “peças” que eu posso usar:

- ▶ **Criacionais:** padrões que propõem soluções flexíveis para criação de objetos
- ▶ **Estruturais:** padrões que propõem soluções flexíveis para composição de classes e objetos
- ▶ **Comportamentais:** padrões que propõem soluções flexíveis para interação e divisão de responsabilidades entre classes e objetos

23 PADRÕES (GOF)

Criacionais

Abstract Factory

Factory Method

Singleton

Builder

Prototype.

Estruturais

Proxy

Adapter

Facade

Decorator

Bridge

Composite

Flyweight

Comportamentais

Strategy

Observer

Template Method

Visitor

Chain of Responsibility

Command

Interpreter

Iterator

Mediator

Memento

State

PADRÕES DE PROJETO

Soluções recorrentes para problemas de projeto enfrentados por desenvolvedores de software

Estrutura da Apresentação dos Padrões

- ▶ Contexto (sistema ou parte de um sistema)
- ▶ Problema (de projeto)
- ▶ Solução (por meio de um padrão de projeto)

IMPORTANTE

Padrões de projeto ajudam em *design for change*

Facilitam mudanças futuras no código

Se o código dificilmente vai precisar de mudar, então uso de padrões é um exemplo de **overengineering**

IMPORTANTE

Padrões de projeto ajudam em *design for change*

Facilitam mudanças futuras no código

Se o código dificilmente vai precisar de mudar, então uso de padrões é um exemplo de **overengineering**

Complementando...

- ▶ Quais são as “peças” que posso usar no design do meu software para torná-lo **flexível e fácil de modificar**?

Padrões de Projeto

Introdução

Padrões

Padrões Criacionais

Padrões Estruturais

Padrões Comportamentais

ESTRUTURA

Relembrando a estrutura da explicação:

- ▶ Contexto
- ▶ Problema
- ▶ Solução

PADRÃO FÁBRICA

Contexto: Sistema que usa canais de comunicação

```
1 void f() {  
2     TCPChannel c = new TCPChannel();  
3     ...  
4 }  
5  
6 void g() {  
7     TCPChannel c = new TCPChannel();  
8     ...  
9 }  
10  
11 void h() {  
12     TCPChannel c = new TCPChannel();  
13     ...  
14 }
```

PADRÃO FÁBRICA

Problema:

Alguns clientes vão precisar de usar UDP, em vez de TCP

Como parametrizar as chamadas de new?

Estão espalhadas em vários pontos do código

Como criar um projeto que facilite essa mudança?

PADRÃO FÁBRICA

Solução: Padrão Fábrica

Fábrica: método que centraliza a criação de certos objetos. Assim temos um único ponto de mudança se quisermos mudar para UDP.

```
1 class ChannelFactory {
2     public static Channel create() { // método fábrica estático
3         return new TCPChannel();
4     }
5 }
6
7 void f() {
8     Channel c = ChannelFactory.create();
9     ...
10 }
11 ...
```

PADRÃO SINGLETON

Contexto: Classe Logger

```
1 void f() {  
2     Logger log = new Logger();  
3     log.println("Executando f");  
4     ...  
5 }  
6 void g() {  
7     Logger log = new Logger();  
8     log.println("Executando g");  
9     ...  
10 }  
11 void h() {  
12     Logger log = new Logger();  
13     log.println("Executando h");  
14     ...  
15 }
```

PADRÃO SINGLETON

Problema: Cada método cliente usa sua própria instância de Logger

- ▶ Como fazer com que todos os clientes usem a mesma instância de Logger?
- ▶ Na verdade, deve existir uma única instância de Logger
- ▶ Todo “log” seria registrado no mesmo arquivo

PADRÃO SINGLETON

Solução: Padrão Singleton

Transformar a classe Logger em um Singleton

Singleton: classe que possui no máximo uma instância

PADRÃO SINGLETON

```
1 class Logger {
2
3     private Logger() {} // proíbe clientes chamar new Logger()
4
5     private static Logger instance; // instância única
6
7     public static Logger getInstance() {
8         if (instance == null) // 1a vez que chama-se getInstance
9             instance = new Logger();
10        return instance;
11    }
12
13    public void println(String msg) {
14        // registra msg no console, mas poderia ser em arquivo
15        System.out.println(msg);
16    }
17 }
```

PADRÃO SINGLETON

```
1 void f() {  
2     Logger log = Logger.getInstance();  
3     log.println("Executando f");  
4     ...  
5 }  
6  
7 void g() {  
8     Logger log = Logger.getInstance();  
9     log.println("Executando g");  
10    ...  
11 }  
12  
13 void h() {  
14     Logger log = Logger.getInstance();  
15     log.println("Executando h");  
16     ...  
17 }
```


Padrões de Projeto

Introdução

Padrões

Padrões Criacionais

Padrões Estruturais

Padrões Comportamentais

PADRÃO PROXY

Contexto: Função que pesquisa livros

```
1 class BookSearch {  
2     ...  
3     Book getBook(String ISBN) { ... }  
4     ...  
5 }
```

PADRÃO PROXY

Problema: Inserir um cache (para melhorar desempenho)

PADRÃO PROXY

Problema: Inserir um cache (para melhorar desempenho)

Se “livro no cache”

- ▶ retorna livro imediatamente
- ▶ caso contrário, continua com a pesquisa

PADRÃO PROXY

Problema: Inserir um cache (para melhorar desempenho)

Se “livro no cache”

- ▶ retorna livro imediatamente
- ▶ caso contrário, continua com a pesquisa

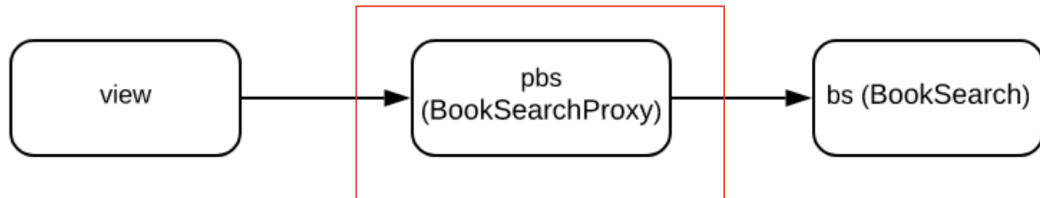
Porém, não queremos modificar o código de BookSearch

- ▶ Classe já está funcionando
- ▶ Já tem um desenvolvedor acostumado a mantê-la, etc

PADRÃO PROXY

Solução: Padrão Proxy

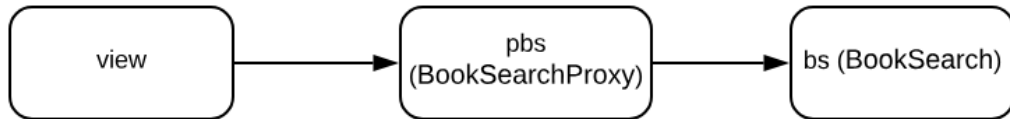
Proxy: objeto intermediário entre cliente e um objeto base
Clientes não “falam” mais com o objeto base (diretamente)
Eles têm que passar antes pelo proxy



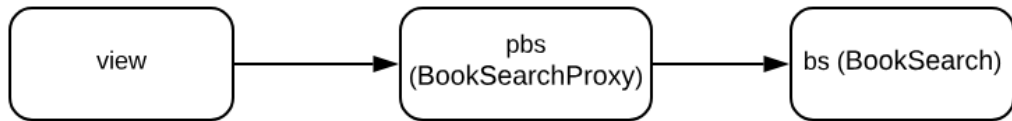
Implementa a lógica do cache

PADRÃO PROXY

```
1 void main() {  
2     BookSearch bs = new BookSearch();  
3     BookSearchProxy pbs;  
4     pbs = new BookSearchProxy(bs);  
5     ...  
6     View view = new View(pbs);  
7     ...  
8 }
```



PADRÃO PROXY



BookSearchProxy: Requisito Não-Funcional

BookSearch: Requisito Funcional

Logo, um proxy:

- 1) ajuda a melhorar qual propriedade de projeto?
- 2) é um aplicação de qual princípio SOLID?

Integridade Conceitual; Information Hiding; Coesão; Acoplamento

Single Responsibility; Open-Closed; Liskov Substitution; Interface Segregation; Dependency Inversion

PADRÃO ADAPTADOR

Contexto: Sistema para Controlar Projetores Multimídia

```
1 class ProjetorSamsung {  
2     public void turnOn() { ... }  
3     ...  
4 }  
5  
6 class ProjetorLG {  
7     public void enable(int timer) { ... }  
8     ...  
9 }
```

Classes fornecidas pelos fabricantes dos projetores

PADRÃO ADAPTADOR

Problema: No sistema de controle de multimídias, eu gostaria de manipular uma única interface Projetor

```
1 interface Projetor {  
2     void liga();  
3 }  
4 ...  
5 class SistemaControleProjetores {  
6     void init(Projetor projetor) {  
7         projetor.liga(); // liga qualquer projetor  
8     }  
9 }
```

PADRÃO ADAPTADOR

```
1 interface Projetor {  
2     void liga();  
3 }  
4 ...  
5 class SistemaControleProjetores {  
6     void init(Projetor projetor) {  
7         projetor.liga(); // liga qualquer projetor  
8     }  
9 }
```

Sendo mais técnico, eu gostaria de aproveitar e fazer uso de qual princípio SOLID?

*Single Responsibility; Open-Closed; Liskov Substitution; Interface Segregation;
Dependency Inversion*

PADRÃO ADAPTADOR

Problema:

São classes dos fabricantes dos projetores

Eu não tenho acesso a elas para, por exemplo, fazer com que elas implementem a interface Projetor

```
1 class ProjetorSamsung {  
2     public void turnOn() { ... }  
3     ...  
4 }  
5  
6 class ProjetorLG {  
7     public void enable(int timer) { ... }  
8     ...  
9 }
```

PADRÃO ADAPTADOR

Solução: Padrão Adaptador

Padrão (ou interface)
de entrada

Padrão (ou interface)
de saída

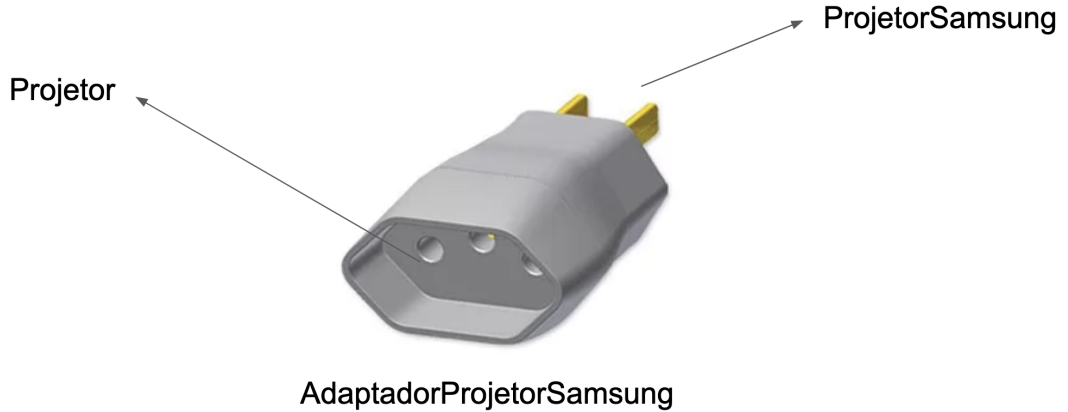


Classe Adaptadora

PADRÃO ADAPTADOR

```
1 class AdaptadorProjetoSamsung implements Projeto {
2
3     private ProjetoSamsung projetor;
4
5     AdaptadorProjetoSamsung (ProjetoSamsung projetor) {
6         this.projetor = projetor;
7     }
8
9     public void liga() {
10         projetor.turnOn();
11     }
12 }
```

PADRÃO ADAPTADOR



PADRÃO FACHADA

Contexto: módulo M usado por diversos outros módulos

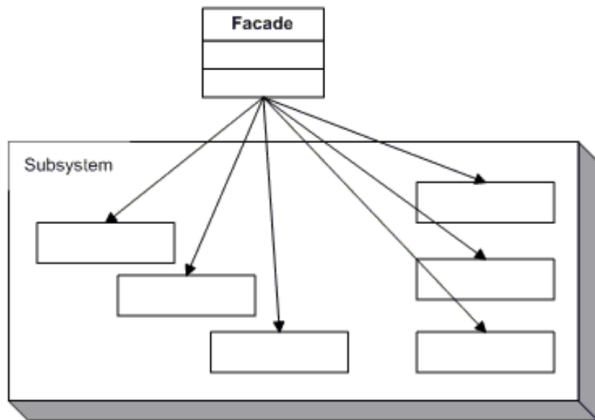
Problema:

- ▶ Interface de M é complexa
- ▶ Clientes reclamam que é difícil usar a interface de M

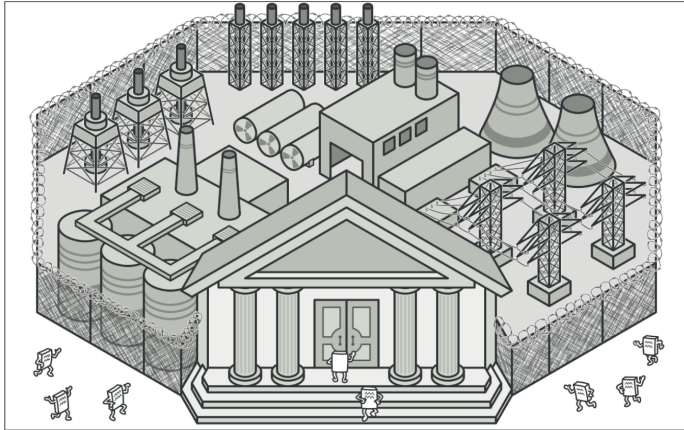
PADRÃO FACHADA

Solução

Criar uma interface mais simples para M, chamada de Fachada.



PADRÃO FACHADA



<https://refactoring.guru/design-patterns/facade>

PADRÃO FACHADA

Exemplo: Interpretador

```
1 Scanner s = new Scanner("prog1.x");  
2 Parser p = new Parser(s);  
3 AST ast = p.parse();  
4 CodeGenerator code = new CodeGenerator(ast);  
5 code.eval();
```

PADRÃO FACHADA

Exemplo: Interpretador

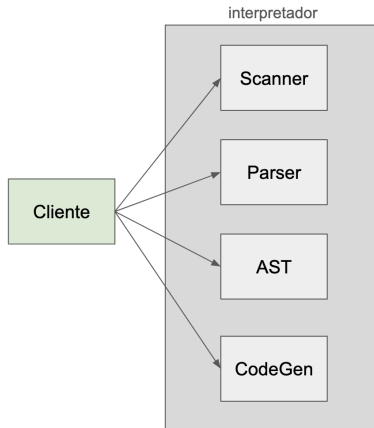
```
1 Scanner s = new Scanner("prog1.x");  
2 Parser p = new Parser(s);  
3 AST ast = p.parse();  
4 CodeGenerator code = new CodeGenerator(ast);  
5 code.eval();
```

```
1 new InterpretadorX("prog1.x").eval();
```

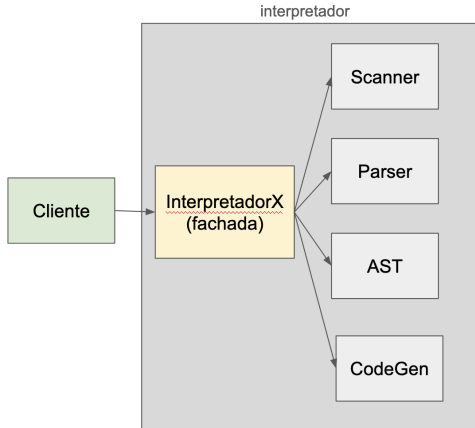
Interface mais simples, que a de cima

PADRÃO FACHADA

Sem Fachada



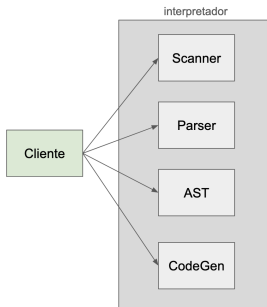
Com Fachada



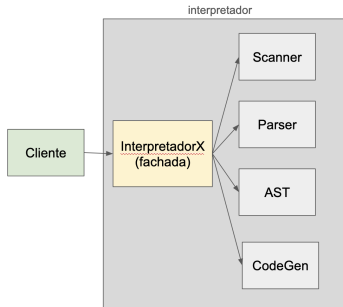
PADRÃO FACHADA

Considerando o cliente, qual propriedade do seu projeto estamos melhorando?

Sem Fachada



Com Fachada

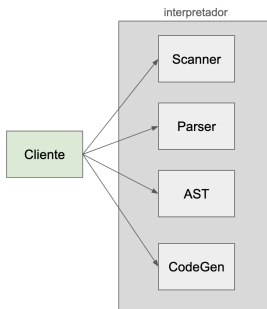


Integridade Conceitual; Information Hiding; Coesão; Acoplamento

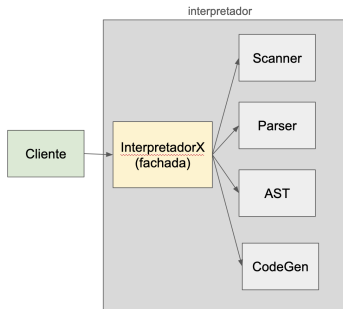
PADRÃO FACHADA

Por outro lado, se não tomarmos cuidado, qual princípio de projeto pode ser violado quando se usa uma fachada?

Sem Fachada



Com Fachada



Single Responsibility; Open-Closed; Liskov Substitution; Interface Segregation; Dependency Inversion

EXERCÍCIOS

- 1 Singleton é um dos padrões de projeto mais criticados e polêmicos.

Erich Gamma, um dos autores do livro GoF, já chegou inclusive a recomendar, em uma entrevista, que ele deveria ser eliminado do catálogo:

Erich: When discussing which patterns to drop, we found that we still love them all. (Not really—I'm in favor of dropping Singleton...)

Descreva porque esse padrão pode causar problemas de design, caso não seja usado de forma adequada.

EXERCÍCIOS

- 2 Por que a implementação de Singleton mostrada nos slides não funciona com sistemas concorrentes (multi-thread)? Descreva um erro que pode ocorrer quando ela é usada com esse tipo de sistema.

EXERCÍCIOS

- ② Por que a implementação de Singleton mostrada nos slides não funciona com sistemas concorrentes (multi-thread)? Descreva um erro que pode ocorrer quando ela é usada com esse tipo de sistema.
- ③ Além de otimização de desempenho via um cache, tal como visto nos slides, descreva outros três requisitos não-funcionais que podem ser implementados em um Proxy.

EXERCÍCIOS

```
1 public class Singleton {
2     private static Singleton instance;
3
4     private Singleton() {}
5
6     public static Singleton getInstance() {
7         if (instance == null) {
8             synchronized (Singleton.class) {
9                 if (instance == null) {
10                     instance = new Singleton();
11                 }
12             }
13         }
14         return instance;
15     }
16 }
```

PADRÃO DECORADOR

Contexto: Sistema que usa canais de comunicação (já foi usado para explicar o padrão Fábrica)

```
1 interface Channel {
2     void send(String msg);
3     String receive();
4 }
5
6 class TCPChannel implements Channel {
7     ...
8 }
9
10 class UDPChannel implements Channel {
11     ...
12 }
```

PADRÃO DECORADOR

Problema: Precisamos adicionar funcionalidades extras em canais

Os canais default (TCP, UDP) não são suficientes

Precisamos também de canais com:

- ▶ Compactação e descompactação
- ▶ Buffers (ou caches)
- ▶ Logging
- ▶ etc

PADRÃO DECORADOR

Solução: Padrão Decorador

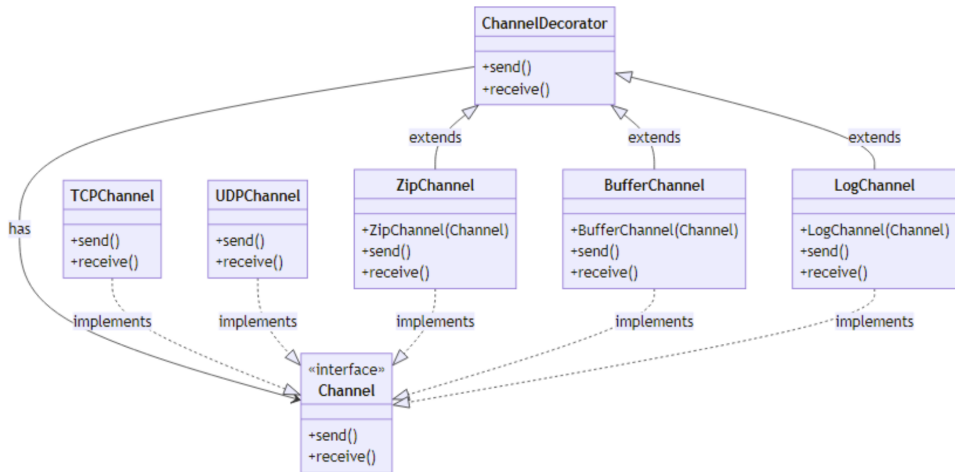
Resolve esse problema – adicionar funcionalidades em uma classe – por meio de composição e sem gerar um número excessivo de classes

```
1 channel = new ZipChannel(new TCPChannel());
2 // TCPChannel que compacte/descompacte dados
3
4 channel = new BufferChannel(new TCPChannel());
5 // TCPChannel com um buffer associado
6
7 channel = new BufferChannel(new UDPChannel());
8 // UDPChannel com um buffer associado
9
10 channel= new BufferChannel(new ZipChannel(new TCPChannel()));
11 // TCPChannel com compactação e um buffer associado
```

PADRÃO DECORADOR



PADRÃO DECORADOR



PADRÃO DECORADOR

```
1 class ChannelDecorator implements Channel {
2
3     private Channel channel;
4
5     public ChannelDecorator(Channel channel) {
6         this.channel = channel;
7     }
8
9     public void send(String msg) {
10         channel.send(msg);
11     }
12
13     public String receive() {
14         return channel.receive();
15     }
16
17 }
```

PADRÃO DECORADOR

```
1 class ZipChannel extends ChannelDecorator {
2     public ZipChannel(Channel c) {
3         super(c);
4     }
5     public void send(String msg) {
6         "compacta mensagem msg"
7         super.send(msg);
8     }
9     public String receive() {
10        String msg = super.receive();
11        "descompacta mensagem msg"
12        return msg;
13    }
14 }
15 ...
16 Channel c = new ZipChannel(new TCPChannel());
17 c.send("Hello, world")
```

Padrões de Projeto

Introdução

Padrões

Padrões Criacionais

Padrões Estruturais

Padrões Comportamentais

PADRÃO STRATEGY

Contexto: Biblioteca de Estruturas de Dados

```
1 class MyList {
2
3     ... // dados de uma lista
4     ... // métodos de uma lista: add, delete, search
5
6     public void sort() {
7         ... // ordena a lista usando Quicksort
8     }
9 }
```

PADRÃO STRATEGY

Problema: Classe MyList *não* está aberta a extensões

Possível extensão: mudar o algoritmo de ordenação

- ▶ Usar ShellSort, HeapSort, etc

PADRÃO STRATEGY

Problema: Classe MyList *não* está aberta a extensões

Possível extensão: mudar o algoritmo de ordenação

- ▶ Usar ShellSort, HeapSort, etc

Solução: Padrão Strategy

Objetivo: Parametrizar os algoritmos usados por uma classe

Tornar uma classe “aberta” a novos algoritmos

No exemplo: novos algoritmos de ordenação

PADRÃO STRATEGY

Passo #1: Criar uma hierarquia de “estratégias” (estratégia = algoritmo)

```
1 abstract class SortStrategy {  
2     abstract void sort(MyList list);  
3 }  
4  
5 class QuickSortStrategy extends SortStrategy {  
6     void sort(MyList list) { ... }  
7 }  
8  
9 class ShellSortStrategy extends SortStrategy {  
10     void sort(MyList list) { ... }  
11 }
```

SortStrategy poderia ser também uma interface

PADRÃO STRATEGY

Passo #2: Modificar MyList para usar a hierarquia de estratégias

```
1 class MyList {
2     ... // dados e métodos
3     private SortStrategy strategy;
4
5     public MyList() {
6         this.strategy = new QuickSortStrategy();
7     }
8
9     public void setSortStrategy(SortStrategy strategy) {
10         this.strategy = strategy;
11     }
12
13     public void sort() {
14         strategy.sort(this);
15     }
16 }
```


PADRÃO OBSERVADOR

Contexto: Sistema de uma Estação Meteorológica

Duas classes principais:

- ▶ Temperatura
- ▶ Termômetro

Diversos termômetros: digital, analógico, web, celular, etc Se a temperatura mudar, os termômetros devem ser atualizados

PADRÃO OBSERVADOR

Problema: Não queremos acoplar Temperatura a Termômetros

Motivos:

- ▶ Tornar classe de dados (modelo) independente de classes de visão (interface com usuários)
- ▶ Tornar flexível a adição de um novo tipo de termômetro no sistema

PADRÃO OBSERVADOR

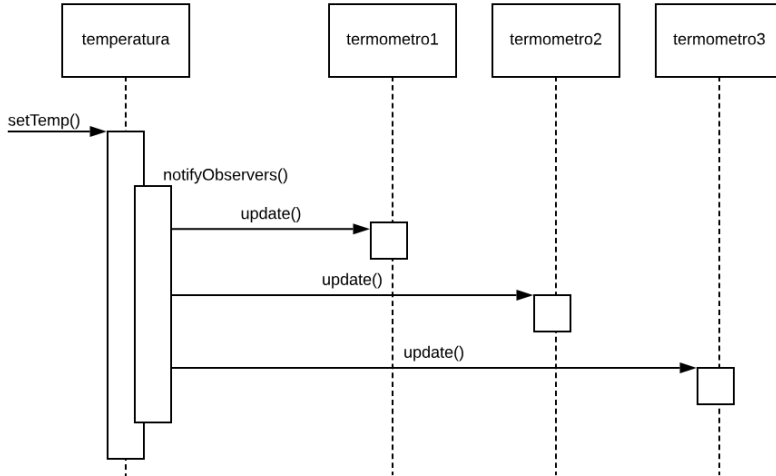
Solução: Padrão Observador

Implementa uma relação do tipo um-para-muitos entre os seguintes objetos

- ▶ Sujeito (Temperatura)
- ▶ Observadores (Termômetros)

Quando o estado de um Sujeito muda, seus Observadores são notificados. Mas Sujeito não conhece o tipo concreto de seus Observadores

PADRÃO OBSERVADOR



PADRÃO OBSERVADOR

Programa Principal com dois observadores

```
1 void main() {  
2     Temperatura t = new Temperatura();  
3     t.addObserver(new TermometroCelsius());  
4     t.addObserver(new TermometroFahrenheit());  
5     t.setTemp(100.0);  
6 }
```

PADRÃO OBSERVADOR

Classe Temperatura

Classe que implementa addObservers e notifyObservers

```
1 class Temperatura extends Subject {  
2  
3     private double temp;  
4  
5     public double getTemp() {  
6         return temp;  
7     }  
8  
9     public void setTemp(double temp) {  
10        this.temp = temp;  
11        notifyObservers();  
12    }  
13 }
```

PADRÃO OBSERVADOR

Uma classe Termômetro

```
1 class TermometroCelsius implements Observer {  
2  
3     public void update(Subject s){  
4         double temp = ((Temperatura) s).getTemp();  
5         System.out.println("Temperatura Celsius: " + temp);  
6     }  
7 }
```

Todos Observadores devem implementar esse método.

Chamada de notifyObservers (em Temperatura) resulta na execução de update de cada um de seus observadores

OUTRO EXEMPLO: PLANILHAS

Quando o valor de uma célula X muda, todas as células Y que fazem uso de X são automaticamente atualizadas

	A	B	C
1	Prova 1	Prova 2	Total
2	80	85	165

Diagram illustrating the Subject-Observer pattern for spreadsheet updates:

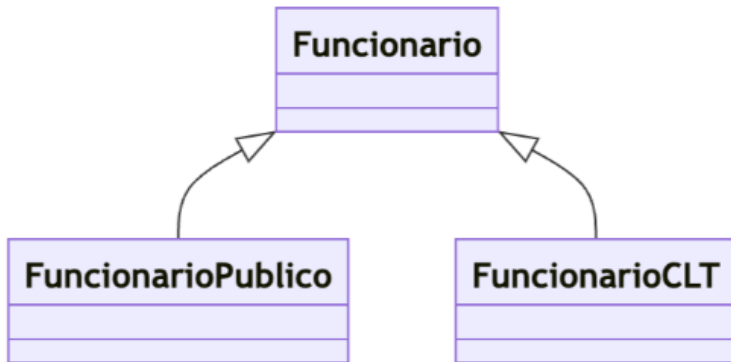
- The **Subject** is represented by the cells **Prova 1** (A2) and **Prova 2** (B2), which are the source of data.
- The **Observer** is represented by the cell **Total** (C2), which depends on the values in **Prova 1** and **Prova 2**.
- Arrows indicate the flow of data: from **Prova 1** and **Prova 2** to the **Subject** label, and from **Total** to the **Observer** label.

OUTRO EXEMPLO: PLANILHAS

```
1 class Cell:
2     def __init__(self):
3         self.value = None
4         self.observers = []
5
6     def set_value(self, new_value):
7         self.value = new_value
8         self.notify_observers()
9
10    def notify_observers(self):
11        for observer in self.observers:
12            observer.update()
13
14    def add_observer(self, observer):
15        self.observers.append(observer)
```

PADRÃO TEMPLATE METHOD

Contexto: Folha de Pagamento



PADRÃO TEMPLATE METHOD

Problema: Função que calcula salário de funcionários:

Passos são semelhantes para funcionários públicos e CLT

Porém, existem alguns detalhes diferentes

Na classe pai (Funcionario) queremos definir o “workflow principal” (ou o template) para cálculo de salários

E deixar aberto para as subclasses os refinamentos desses passos

PADRÃO TEMPLATE METHOD

Solução: Padrão Template Method

Especifica como implementar o esqueleto de um algoritmo em uma classe abstrata X

Mas deixando pendente alguns passos (ou métodos abstratos) para serem implementados nas subclasses

Permite que subclasses customizem um algoritmo, mas sem mudar sua estrutura

PADRÃO TEMPLATE METHOD

```
1 abstract class Funcionario {
2
3     double salario;
4     ...
5     abstract double calcDescontosPrevidencia();
6     abstract double calcDescontosPlanoSaude();
7     abstract double calcOutrosDescontos();
8
9     public double calcSalarioLiquido() { // template method
10         double prev = calcDescontosPrevidencia();
11         double saude = calcDescontosPlanoSaude();
12         double outros = calcOutrosDescontos();
13         return salario - prev - saude - outros;
14     }
15 }
```

INVERSÃO DE CONTROLE

Template Method é usado para implementar Inversão de Controle, principalmente em frameworks

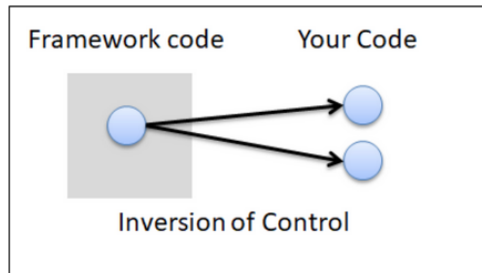
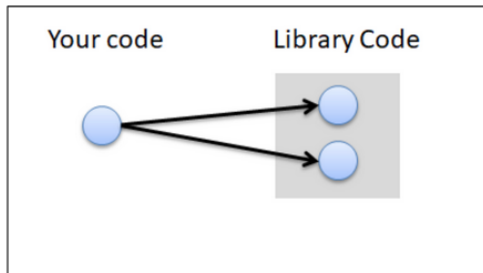
Framework: define o “modelo” de um algoritmo/sistema

Mas clientes podem parametrizar alguns passos

Framework chama esse código dos clientes

Daí o termo inversão de controle

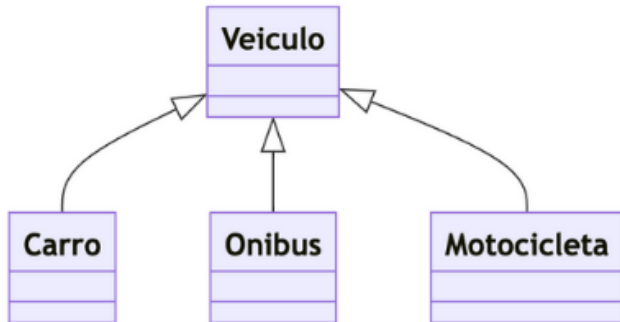
FRAMEWORKS VS BIBLIOTECAS



Fonte: <https://github.com/prmr/SoftwareDesign/blob/master/modules/Module-06.md>

PADRÃO VISITOR

Contexto: Veiculo e subclasses



PADRÃO VISITOR

Imagina uma lista polimórfica de Veículos

```
1 List<Veiculo> list = new ArrayList<Veiculo>();  
2 list.add(new Carro(..));  
3 list.add(new Onibus(..));  
4 ...
```

Por que essa lista é chamada de polimórfica?

PADRÃO VISITOR

Problema: Temos que realizar certas operações com os veículos da lista

Exemplo:

- ▶ Imprimir dados de veículos (sejam Carro, Onibus,...)
- ▶ Salvar dados de veículos em disco
- ▶ Enviar uma msg para os donos dos veículos
- ▶ etc

PADRÃO VISITOR

Queremos seguir o princípio Aberto/Fechado

- ▶ Manter Veiculo e suas subclasses fechados para mudanças, mas aberto para extensões
- ▶ Extensões: operações que queremos realizar com veículos

PADRÃO VISITOR

Possível alternativa

```
1 interface Visitor {
2     void visit(Carro c);
3     void visit(Onibus o);
4     void visit(Motocicleta m);
5 }
6
7 class PrintVisitor implements Visitor {
8     public void visit(Carro c) { "imprime dados de carro" }
9     public void visit(Onibus o) { "imprime dados de onibus" }
10    public void visit(Motocicleta m) {"imprime dados de moto"}
11 }
```

Interface foi criada porque amanhã poderemos ter outros tipos de Visitors (exemplo: que salva os dados em um arquivo JSON)

PADRÃO VISITOR

```
1 PrintVisitor visitor = new PrintVisitor();  
2 foreach (Veiculo veiculo: listaDeVeiculosEstacionados) {  
3     visitor.visit(veiculo); // erro de compilação  
4 }
```

Em Java e linguagens similares, compilador não conhece o tipo dinâmico do parâmetro veiculo

Logo, ele não sabe identificar qual dos três métodos visit de PrintVisitor deverá ser chamado

PADRÃO VISITOR

Decisão do método a ser chamado é “bi-dimensional”

	PrintVisitor	MailVisitor	SaveVisitor	etc
Carro				
Onibus				
Motocicleta				
etc				

PADRÃO VISITOR

Solução: Padrão Visitor

Permite adicionar uma operação genérica em uma família de classes, sem mexer no código delas

Simula multiple dispatching, em linguagens que oferecem apenas single dispatching

PADRÃO VISITOR

```
1 abstract class Veiculo {
2     abstract public void accept(Visitor v);
3 }
4 class Carro extends Veiculo {
5     ...
6     public void accept(Visitor v) {
7         v.visit(this);
8     }
9     ...
10 }
11 class Onibus extends Veiculo {
12     ...
13     public void accept(Visitor v) {
14         v.visit(this);
15     }
16     ...
17 }
18 // Idem para Motocicleta
```


PADRÃO VISITOR

```
1 PrintVisitor visitor = new PrintVisitor();
2 foreach (Veiculo veiculo: listaDeVeiculosEstacionados) {
3     veiculo.accept(visitor);
4 }
```

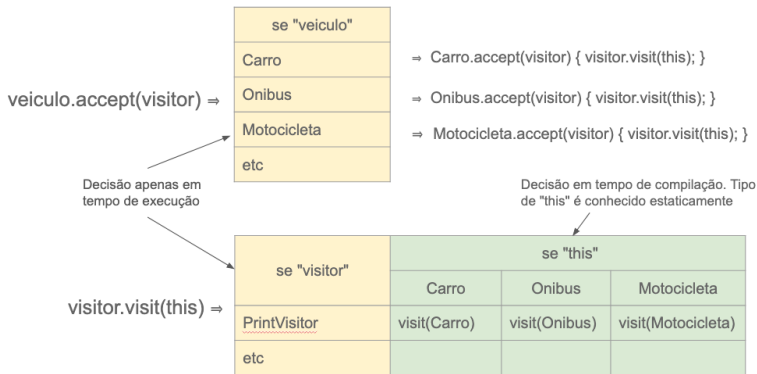
Chama accept do tipo dinâmico de veiculo. Suponha que seja Carro

```
1 class Carro extends Veiculo {
2     ...
3     public void accept(Visitor v) {
4         v.visit(this);
5     }
6     ...
7 }
```

Tipo de this é conhecido em tempo de compilação (Carro)

PADRÃO VISITOR

A escolha do método ocorre em dois passos dinâmicos, mais um passo estático



PADRÃO VISITOR

Vantagens

- ▶ Visitors facilitam a adição de um método em uma hierarquia de classes
- ▶ Pode existir um segundo Visitor, com outras operações
- ▶ Exemplo: calcular imposto de veículos

Desvantagens

- ▶ A adição de uma nova classe na hierarquia (exemplo: Caminhao), implica que todos os Visitors terão que ser atualizados, com um novo método: visit(Caminhao)
- ▶ Visitors podem quebrar encapsulamento. Por exemplo, Veiculo pode ter que implementar métodos públicos expondo seu estado interno para que os Visitors tenham acesso a ele

Quando não vale a pena usar padrões de projeto?

PADRÕES DE PROJETO <-> DESIGN FOR CHANGE

Se a chance de mudanças é zero, não precisamos de padrões de projeto

Para viabilizar “design for change”, Design Patterns complicam um pouco o projeto

Por exemplo, demandam a criação de classes extras

EXEMPLO: SOLUÇÃO SEM PADRÃO DE PROJETO STRATEGY

```
1 class MyList {  
2     ... // dados de uma lista  
3     ... // métodos de uma lista: add, delete, search  
4     public void sort() {  
5         ... // ordena a lista usando Quicksort  
6     }
```

OLUÇÃO USANDO STRATEGY

```
1 class MyList {
2     ... // dados de uma lista
3     ... // métodos de uma lista: add, delete, search
4     private SortStrategy strategy;
5     public MyList() { strategy = new QuickSortStrategy(); }
6     public void setSortStrategy(SortStrategy strategy) { this.strategy = strategy; }
7     public void sort() {strategy.sort(this);}
8 }
9
10 abstract class SortStrategy {
11     abstract void sort(MyList list);
12 }
13 class QuickSortStrategy extends SortStrategy {
14     void sort(MyList list) { ... }
15 }
16 class ShellSortStrategy extends SortStrategy {
17     void sort(MyList list) { ... }
18 }
```

EXERCÍCIO

- 1) Cite três design patterns que ajudam nitidamente a tornar uma classe “aberta” para extensões? Isto é, favorecem um design que segue o Princípio Aberto/Fechado.

EXERCÍCIO

1) Cite três design patterns que ajudam nitidamente a tornar uma classe “aberta” para extensões? Isto é, favorecem um design que segue o Princípio Aberto/Fechado.

- ▶ Template Method
- ▶ Decorador
- ▶ Visitor
- ▶ Strategy
- ▶ Fábrica

Este documento está licenciado sob a Licença Atribuição-NãoComercial-Compartilhalgual 4.0 Internacional Creative Commons. Para visualizar uma cópia desta licença, visite <http://creativecommons.org/licenses/by-nc-sa/4.0/> ou mande uma carta para Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

